CANDIDATE TECHNICAL ASSESSMENT

Junior – Mid Level Developer

AI Planning Copilot

for Shop Floor Scheduling

Agentic AI + GenAI | Open-Source Stack

Python or C#	Ollama (local LLM)	Agentic AI	PostgreSQL

Duration: 3-5 Days · No paid API keys required · Seed data provided

1. What This Test Is About

We are building AI-powered features on top of an existing manufacturing scheduling system. We need engineers who can connect GenAI and Agentic AI to real data — not just call an LLM API, but design systems where the AI reasons, picks the right tools, queries a database, and explains its answers clearly.

This test simulates that work. You will build a Planning Copilot that sits on top of a PostgreSQL scheduling database. A planner types a question in plain English; your system figures out what to do, retrieves the right data, and returns a clear, trustworthy answer.

> *This test is for junior-to-mid level developers. You are not expected to build a production system. Clean logic, honest explanations, and thoughtful design matter more than feature count.*

1.1 The Two AI Concepts You Must Demonstrate

Concept	What It Means	How It Shows in This Project
GenAI	Using a Large Language Model to understand natural language, generate SQL, and write human-readable explanations.	The /ask endpoint uses an LLM to interpret a plain-English question, generate a SQL query, and explain the result in plain language.
Agentic AI	The AI does not answer in one step. It decides which tool to use, executes it, and may chain multiple steps together.	Your agent selects from tools: run_sql, check_load, simulate_downtime, get_priority, recommend. It picks the right one(s) automatically per question.

1.2 Scoring Overview

#	Dimension	Key Question	Points
1	**Agentic Tool Selection**	Does the agent pick the right tool for each question?	**30**
2	**GenAI / LLM Usage**	Are prompts well-designed? Is the SQL correct and safe?	**25**
3	**Business Logic**	Are delay detection, load calc, and simulation correct?	**25**
4	**Code & Documentation**	Is the code readable? Does the README explain your thinking?	**20**

2. Language & Technology Stack

You may implement this project in either Python or C#. Both are first-class choices. Pick whichever you are more comfortable with — we work with both in production.

2.1 Language Options

Language	API Framework	Notes
Python	FastAPI (recommended)	Fast to scaffold, auto-generates Swagger docs. Good LLM SDK support via openai, ollama, langchain packages.
C#	ASP.NET Core Web API	Strongly typed, familiar to .NET developers. Use Microsoft.SemanticKernel or OllamaSharp for LLM integration. Swagger via Swashbuckle.

Whichever language you choose, the API contract (endpoints, request/response JSON) must be identical. The test questions and evaluation are language-agnostic.

2.2 LLM — Open-Source, Free, No Account Needed

All LLM options below are free and open-source. You do not need a paid API key to complete this test.

Tool	Model	How to Run	Notes
Ollama	Llama 3.2 3B / 8B Mistral 7B Gemma 2 9B	docker run + ollama pull	Recommended. Runs 100% locally inside Docker. No account, no key, works offline. Llama 3.2 3B works well on 8 GB RAM.
Groq API	Llama 3.1 8B Mixtral 8x7B	Free account + API key	Free tier is generous (14,400 req/day). Fastest inference available. Good fallback if local GPU is limited.
HuggingFace Inference API	Mistral 7B Instruct Zephyr 7B	Free account + API key	Free tier available. Slightly slower. Useful if Ollama setup has issues.
LM Studio	Any GGUF model	Desktop app, local server	Alternative to Ollama for local inference. Exposes an OpenAI-compatible REST endpoint on localhost.

2.3 Recommended: Ollama in Docker

Add this to your docker-compose.yml to include Ollama alongside your API and PostgreSQL:

```
services:

  ollama:
    image: ollama/ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama

  # After containers start, pull a model once:
  #   docker exec ollama ollama pull llama3.2
  #   docker exec ollama ollama pull mistral

  api:
    build: .
    environment:
      - OLLAMA_BASE_URL=http://ollama:11434
      - LLM_MODEL=llama3.2
    depends_on:
      - ollama
      - db

volumes:
  ollama_data:
```

2.4 Calling Ollama from Python

```
# pip install ollama
import ollama

response = ollama.chat(
    model='llama3.2',
    messages=[
        {'role': 'system', 'content': system_prompt},
        {'role': 'user',   'content': user_question},
    ]
)
answer = response['message']['content']
```

2.5 Calling Ollama from C#

```
// dotnet add package OllamaSharp
using OllamaSharp;

var ollama = new OllamaApiClient("http://ollama:11434");
var chat   = new Chat(ollama, systemPrompt);

var reply  = await chat.SendAsync(userQuestion);
Console.WriteLine(reply);
```

Ollama also exposes an OpenAI-compatible endpoint at /v1/chat/completions. Any library that supports the OpenAI SDK (Python openai package, C# Azure.AI.OpenAI) can point to Ollama by setting base_url to http://ollama:11434/v1 and api_key to any non-empty string.

2.6 Full Stack Summary

Layer	Python Stack	C# Stack
API Framework	FastAPI	ASP.NET Core 8 Web API
LLM	Ollama (local) or Groq	Ollama via OllamaSharp or OpenAI-compat endpoint
Agent Framework	LangChain / plain Python	Microsoft Semantic Kernel / plain C#
Database	PostgreSQL 14+ via psycopg2 / SQLAlchemy	PostgreSQL 14+ via Npgsql / Entity Framework Core
Containers	Docker + docker-compose	Docker + docker-compose
Tests	pytest	xUnit or NUnit
API Docs	Built-in Swagger via FastAPI	Swashbuckle (auto-enabled in .NET 8 template)

3. The Scenario

A factory runs 8 machines across 4 bays. Each day planners track 20+ work orders. Some machines are broken, some are overloaded, and some orders are already late. The planner opens a chat interface and asks questions in plain English — your copilot answers them.

3.1 Database Tables (provided in seed.sql)

Table	Key Columns	Purpose
machines	machine_id, capacity_hours_day, current_status, available_hours_today	Each machine with its daily capacity and current availability
work_orders	wo_id, product_code, required_machine, processing_time_hr, priority (1–5), due_date, status	All production orders — pending, in_progress, delayed, completed, on_hold
downtime_schedule	machine_id, downtime_date, start_time, end_time, downtime_type	Planned and unplanned machine downtime windows
products	product_code, product_name, product_family	Product catalogue for readable answers
agent_action_log	session_id, action_type, sql_generated, result_summary, confidence	Your agent must write here after every action — audit trail

Three pre-built views are included to help you get started:

- v_machine_load — total queued hours vs capacity per machine, with load %
- v_at_risk_orders — orders that cannot complete given current machine state
- v_priority_queue — orders due within 3 days, ranked by priority

3.2 Interesting Situations in the Seed Data

The data is deliberately designed with six problems your agent must detect and reason about:

Situation	Affected	What the Agent Should Say
Machine completely broken	M4 — unavailable all day	WO-1003 and WO-1005 (both P1) are blocked; escalation needed
Planned maintenance cuts capacity	M2 — only 4.5 h today	WO-1008 (5.5 h) cannot finish today; WO-1009 already delayed
Machine nearly full today	M3 — WO-1002 uses 9 of 10 h	WO-1007 (P1, 7 h) cannot start today — at risk of missing tomorrow
Two P1 orders competing for same machine	M6 — demand 7.5 h vs 6 h cap	M6 overloaded; agent should suggest which order to slip
Tight but feasible window	M8 — available from 11:00, 3 h left	WO-1012 (2.5 h) fits if started immediately at 11:00
Borderline overload with easy fix	M5 — 8 h demand vs 7.5 h cap	Reschedule low-priority WO-1013 to free capacity for P1 WO-1019

4. What to Build

Build a REST API with an AI agent backend. The core is one endpoint that accepts a natural-language question. The agent behind it decides how to answer — it does not just forward the question to the LLM blindly.

4.1 Agent Flow

```
User question
    |
    ▼
[1] Classify ── Is this a planning question? If not, refuse politely.
    |
    ▼
[2] Plan ──────── Which tool is needed?
                    run_sql          → data lookup
                    check_load       → machine capacity
                    simulate         → 'what if' downtime
                    get_priority     → show high-priority / due-soon orders
                    recommend        → 'what should I do' questions
    |
    ▼
[3] Execute ─── Call the selected tool, get data from DB
    |
    ▼
[4] Explain ─── Ask LLM to write a plain-English explanation
    |
    ▼
[5] Log ────────── Write row to agent_action_log
    |
    ▼
Return JSON response
```

Junior tip: Start with a simple if/else tool selector driven by keywords. Once it works for all 6 questions, you can upgrade to LLM-driven tool selection as a bonus.

4.2 Required API Endpoints

Method	Path	Input	Output
POST	/api/v1/ask	{ "question": "..." }	answer, explanation, sql_used, data, tool_used, confidence
GET	/api/v1/machines/load	—	All machines with queued hours, capacity, load %
POST	/api/v1/simulate/downtime	{ "machine_id", "downtime_hours" }	Affected WOs, estimated new delays
GET	/api/v1/orders/at-risk	—	Orders that cannot complete on time, with reason
GET	/api/v1/health	—	Service status + DB connection check

4.3 Response Format for /ask

```
{
  "question":    "Which work orders are delayed?",
  "tool_used":   "run_sql",
  "sql_used":    "SELECT wo_id, status, due_date FROM work_orders WHERE ...",
  "data":        [{ "wo_id": "WO-1003", "status": "delayed", ... }],
  "answer":      "There are 3 delayed orders: WO-1003, WO-1005, WO-1009.",
  "explanation": "WO-1003 and WO-1005 are blocked because machine M4 broke
                  down this morning. WO-1009 missed its window due to M2's
                  planned maintenance reducing available hours to 4.5.",
  "confidence":  0.92,
  "follow_ups":  ["Which machines are causing the most delays?",
                  "Can WO-1003 be rerouted to another machine?"]
}
```

The explanation must read like a planner wrote it — plain language, specific order IDs and machine names, no developer jargon.

4.4 The Six Test Questions

#	Question	Tool Expected	Key Expected Facts
1	Which work orders are delayed?	run_sql	WO-1003, WO-1005, WO-1009
2	Which machines are overloaded?	check_load	M3 (190%), M6 (125%), M5 (106%)
3	Why is WO-1003 at risk?	run_sql + explain	M4 is unavailable — order is blocked
4	What happens if M2 is down for 4 extra hours?	simulate	WO-1008 misses due date; WO-1009 deepens
5	Show high-priority orders due this week.	get_priority	WO-1001/2/3/5/7/15 — P1 and P2 this week
6	Recommend actions to reduce delays.	recommend	Fix M4, reroute WO-1009, reschedule WO-1013

4.5 Business Logic to Implement in Code

These calculations must live in your application code — not be generated by the LLM each time:

Delay detection

- status = 'delayed' → already flagged
- status IN ('pending','in_progress') AND due_date <= today → treat as at risk
- status = 'pending' AND machine.current_status = 'unavailable' → blocked

Machine load calculation

- queued_hours = SUM(processing_time_hr) for all pending/in_progress orders on that machine
- load_pct = queued_hours / capacity_hours_day × 100
- Overloaded = load_pct > 100 | At risk = load_pct > 85

Downtime impact simulation

- new_available = available_hours_today − downtime_hours (minimum 0)
- Flag any pending/in_progress order where processing_time_hr > new_available as newly at risk
- Estimated extra delay = processing_time_hr − new_available hours

5. Prompt Engineering Guide

How you write your LLM prompts significantly affects answer quality. Here are the two prompts your system needs, with templates you can adapt.

5.1 System Prompt — SQL Generation

Inject this once at the start of every LLM call that generates SQL:

```
You are a SQL assistant for a manufacturing shop floor scheduling system.

Database schema:
  work_orders(wo_id, product_code, required_machine, processing_time_hr,
              priority, due_date, status, quantity)
  machines(machine_id, machine_name, capacity_hours_day,
           current_status, available_hours_today)
  downtime_schedule(machine_id, downtime_date, start_time, end_time,
                    duration_hours, downtime_type)
  products(product_code, product_name)

Rules:
  1. Return a valid PostgreSQL SELECT query only. No INSERT/UPDATE/DELETE.
  2. Never concatenate user input into SQL. Use $1/$2 placeholders.
  3. Only answer questions about scheduling, work orders, or machines.
  4. If the question is outside scope, reply: OUT_OF_SCOPE
```

5.2 Explanation Prompt

Call this after running the SQL and getting results:

```
You are a planning assistant explaining data to a factory floor manager.

Question asked: {question}
Data returned:  {json_data}

Write a clear explanation in plain English (3–4 sentences).
  - Mention specific order IDs and machine names.
  - If there is a risk or delay, state the cause.
  - Do not use technical terms like 'query', 'JSON', or 'database'.
  - Do not invent data that is not in the results above.
```

Key principle: the LLM must only explain data you have already retrieved from the database. It must never invent machine IDs, work order numbers, or dates.

6. Deliverables & Scoring

6.1 Required Files

File / Folder	What It Must Contain
README.md	How to run, which LLM model you used, any known limitations, and at least one example question with its full JSON response
docker-compose.yml	Starts API + PostgreSQL + Ollama. seed.sql runs automatically. docker-compose up --build must be the only command needed.
seed.sql	Use the provided seed.sql as-is. You may add to it but must not rename tables or columns.
.env.example	Template for env vars (DB_URL, OLLAMA_BASE_URL, LLM_MODEL, etc.). Never commit real secrets.
src/ (Python) or App/ (C#)	Source code with clear module separation: api/routes, agent/tools, db/queries, models/schemas
tests/	Tests covering: delay detection function, machine load calculation, /api/v1/ask response schema
DESIGN.md (optional)	What trade-offs did you make? What would you do differently with more time? Shows engineering thinking.

6.2 Submission Checklist

1. docker-compose up --build starts with no errors and no manual steps
2. POST /api/v1/ask returns valid JSON with all required fields
3. All six test questions return a plausible answer
4. agent_action_log receives a new row after each /ask call
5. Tests pass (minimum 3 tests)
6. No API keys or secrets in the repository
7. README explains how to run and which LLM model you used

6.3 Scoring Detail

What We Check	Points	How We Test It
Agent picks the correct tool for all 6 test questions	**30**	Inspect tool_used in each response
SQL generated is correct, safe, and parameterised	**15**	Read sql_used; test edge-case questions
Explanation is clear and mentions specific WOs / machines	**10**	Manual review of explanation field
Delay detection, load calc, simulation logic are correct	**25**	Unit tests + API responses vs expected answers
Code is readable, README is clear	**10**	Code review
agent_action_log rows written with correct fields	**5**	SELECT * FROM agent_action_log after run
Bonus: LLM-driven tool selection, multi-turn, streaming, UI	**+10**	Demonstrated during review

7. Tips & Suggested Build Order

Here is a day-by-day approach that many junior candidates find helpful:

Day	Focus	Goals
1	**Foundation**	Get Docker + PostgreSQL + Ollama running. Confirm seed.sql loads. Build /health and /machines/load. Call Ollama manually from code and confirm you get a response for one test question.
2	**Agent + Logic**	Build the tool selector (if/else is fine). Implement delay detection, load calc, and simulation logic in code. Get all 6 test questions returning sensible answers. Write to agent_action_log.
3	**Polish**	Write pytest/xUnit tests. Improve explanation prompt quality. Clean up README. Bonus: upgrade tool selector to use the LLM, add multi-turn support, or add a simple chat UI.

Common Questions

Q: I have never used Ollama before. How hard is it to set up?

Very easy. Add the ollama/ollama image to docker-compose, run docker exec ollama ollama pull llama3.2 once, and it is ready. The Python ollama package or the OpenAI-compat endpoint makes calling it straightforward.

Q: Which model should I use with Ollama?

Llama 3.2 3B is a good starting point — it runs on 8 GB RAM and handles structured SQL generation well. If you have more RAM, try Llama 3.2 8B or Mistral 7B for better reasoning quality.

Q: Can I use Semantic Kernel in C# instead of a manual agent?

Yes. Semantic Kernel's function-calling and planner features map well to this project. Define each tool (run_sql, simulate, etc.) as a Kernel function and let the planner select them. Document your approach in README.

Q: Should the LLM generate the SQL or should I write it?

For the /ask endpoint: the LLM should generate SQL from the question — that is the GenAI part. For the other four endpoints (/machines/load, /simulate/downtime, etc.) you write the SQL directly.

Q: What if I do not finish everything?

Submit what you have. Use DESIGN.md to explain what you would complete next. A well-reasoned partial solution scores better than a rushed complete one with messy code.

Good luck!

We are looking for clear thinking, honest code, and practical AI — not perfection.